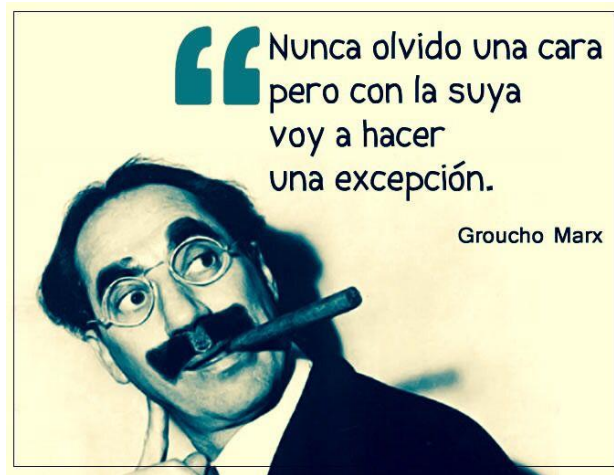


UT-09: CONTROL Y MANEJO DE EXCEPCIONES

¿Qué vamos a ver?

- Excepciones. Concepto y Jerarquías de excepciones.
- Manejo de excepciones:
 - Captura de excepciones.
 - Propagar excepciones.
 - Lanzar excepciones.
- Crear excepciones.
- Aserciones

1. EXCEPCIONES. CONCEPTO Y JERARQUÍA DE EXCEPCIONES.



Los errores en los programas son prácticamente inevitables, tanto si están originados por códigos deficientes, entrada de datos, parámetros incorrectos o archivos inexistentes...

En la mayoría de los lenguajes de programación, la manipulación de los errores suele ser complicada y confusa. Su código se mezcla con el del resto del programa, haciéndolo poco claro, y suele estar destinado solo a evitar el error y no a controlar la situación una vez que dicho error se ha producido.

Java es un lenguaje que se enfoca en tratar de evitar, en la medida de lo posible, que el programa se interrumpa a causa de un error. Para ello no basta con evitar que se produzcan los errores, sino que hay que implementar los medios para que un programa se recupere de las condiciones generadas por un error que no se ha podido evitar. Esto depende del tipo de error, y no siempre es posible.

Cuando, en la ejecución de un programa, se produce una situación anormal, es decir, un error, que interrumpe el flujo normal de ejecución, el método que se esté ejecutando genera un objeto de la clase **Throwable** ("arrojable"), que contiene información del error, de su causa y del contexto del programa en el momento

en que se produce, y lo entrega (“lo arroja”) al sistema de tiempo de ejecución (la máquina virtual). Este objeto es susceptible de ser capturado por el programa y analizarlo para dar una respuesta, si procede. En caso contrario, el programa se interrumpe y el sistema de tiempo de ejecución muestra una serie de mensajes que describen el error, como ocurre en otros lenguajes de programación.

Hay errores lo bastante graves para que sea preferible que el programa se interrumpa. Por ejemplo, errores derivados de problemas de hardware. Si intentamos leer de un disco defectuoso, se producirán errores de los que es difícil, si no imposible, recuperarse. Estos y otros errores relacionados directamente con la máquina virtual, y no con el código de nuestro programa, arrojan objetos de la clase Error, una subclase de Throwable. De ellos no nos vamos a ocupar, ya que poco se puede hacer con estos errores a la hora de programar. Tampoco nos ocuparemos de las excepciones llamadas de tiempo de ejecución (runtime exceptions), que proceden de líneas de código equivocadas. La solución a estos errores es corregir el código.

Pero hay otras clases de errores más habituales y menos graves, como entradas de datos de tipos equivocados, aperturas de ficheros con ruta de acceso errónea u operaciones aritméticas no permitidas. A estos errores se les llama **excepciones**, y producen un objeto de la clase Exception, otra subclase de Throwable. A continuación, explicaremos estas excepciones, ya que son manipulables a través del código. Para ello se usan los bloques try, catch y finally.



Podemos agrupar las excepciones en dos grupos:

- **Excepciones comprobadas**
- **Excepciones no comprobadas**

Entre las **excepciones comprobadas** se encuentran las más comunes, generalmente con un origen fuera del programa (entradas de datos, nombres de ficheros incorrectos, etc). El compilador no exige su tratamiento, por tanto, no tenemos que preocuparnos por saber cuáles son exactamente, aunque con la práctica acabaremos familiarizándonos con las más importantes: **IOException**, **FileNotFoundException**, **NumberFormatException**, ...

Con respecto a las **excepciones no comprobadas**, como **ArithmeticException**, producidas por errores aritméticos, o **ArrayIndexOutOfBoundsException**, que se produce cuando intentamos salirnos de los límites de un array. Dichas excepciones suelen estar asociadas a malas prácticas de programación. Por tanto, más que tratarlas con una estructura **try-catch**, hay que corregir el código para evitar que se produzcan.

2. MANEJO DE EXCEPCIONES

2.1. Captura de excepciones

Cuando sabemos que en un determinado fragmento de código se puede producir una excepción, lo encerramos dentro de un bloque try. Por ejemplo, si en una división entre las variables a y b sospechamos que divisor b podría hacerse cero en algún momento escribimos:

```
try{  
    int c;  
    c = a / b;  
}
```

Con esta estructura estamos sometiendo a observación al bloque de código cerrado entre llaves. Si salta una excepción en ese bloque, deberá ser capturada por un bloque `catch` de la siguiente forma:

```
try{
    int c;
    c = a / b;
} catch(ArithmeticException e){
    System.out.println("Error: división por 0");
}
```

Si se produce una excepción y es capturada, se interrumpe la ejecución del código del bloque **try**, saltando a la primera línea del bloque **catch**. Cuando se termina de ejecutar dicho bloque, continúa en la línea inmediatamente posterior a la estructura **try-catch**.

La palabra reservada **catch** va seguida de unos paréntesis donde se encierra un parámetro de la clase de excepción que puede atrapar; en este caso, una excepción aritmética. El parámetro `e` se puede usar como variable local dentro del bloque. Hace referencia al objeto de la excepción y contiene toda la información sobre el error que la ha producido. Entre sus métodos está **getMessage()**, que nos muestra un mensaje descriptivo del error.

Podríamos haber puesto

```
try{
    int c;
    c = a / b;
} catch(ArithmeticException e){
    System.out.println(e.getMessage());
}
```

O incluso

```
try{
    int c;
    c = a / b;
} catch(ArithmeticException e){
```

```
System.out.println(e);  
}
```

Ya que hace una llamada a `toString()` de la clase de la variable `e`, donde también se describe el error (en inglés).

En el bloque `catch` solo se recogerán las excepciones del tipo declarado entre paréntesis o de una subclase. En el ejemplo anterior podríamos haber escrito lo siguiente:

```
try{  
    int c;  
    c = a / b;  
} catch(Exception e){  
    System.out.println(e);  
}
```

Ya que **`ArithmeticException`** es una subclase de **`Exception`**.

Esto nos permitirá recoger otros tipos de excepción en el mismo `catch`, pero, para ese caso, es mejor escribir más de un bloque `catch`. Con un mismo bloque `try` se pueden poner tantos bloques `catch` como deseemos, siempre que vayan seguidos,

```
try{  
    //.bloque try  
} catch (tipoExcepcional1 nombreParametro1){  
    //.bloque catch  
} catch (tipoExcepcional2 nombreParametro2){  
    ...  
}
```

Cuando se produce una excepción en el bloque `try`, se compara con el tipo del primer bloque `catch`. Si coincide con él o con una subclase, se ejecuta dicho bloque y continúa el programa después del último bloque `catch`. Si no coincide, se compara con el tipo del segundo bloque, y así sucesivamente hasta que se encuentra un bloque cuyo parámetro coincida o sea una superclase de la

excepción producida, de forma que solo se ejecuta un bloque catch, el primero cuyo tipo sea compatible. Si la excepción no coincide con ninguno de los parámetros de los bloques, no es capturada y se interrumpe la ejecución del programa. Aquí hay que tener cuidado de no poner un bloque catch con una excepción que sea superclase de otra que vaya más abajo, pues el bloque de esta última nunca se ejecutará.

Por ejemplo:

```
try{
    int c;
    c = a / b;
} catch (Exception e){
    System.out.println("Estoy en el primer catch");
} catch(ArithmeticException e) {
    System.out.println("Estoy en el segundo catch");
}
```

Si b vale cero, se producirá una excepción de tipo ArithmeticException, pero, al ser un subtipo de Exception, será capturada en el primer catch, cuyo bloque será el que se ejecute, apareciendo el mensaje “Estoy en el primer catch”. La ejecución seguirá después del último bloque, de modo que el segundo bloque catch es inútil, ya que jamás se va a ejecutar. De hecho, en este ejemplo, como todo tipo de excepción es subclase de Exception, cualquier excepción que se produzca en el bloque try será capturada en el primer bloque catch y ningún bloque que pongamos después se va a ejecutar nunca.

Por otra parte, existe la posibilidad de capturar más de un tipo de excepción con un único bloque catch.

```
catch (tipoExcepcion1 | tipoExcepcion2 | ... e){
    ...
}
```

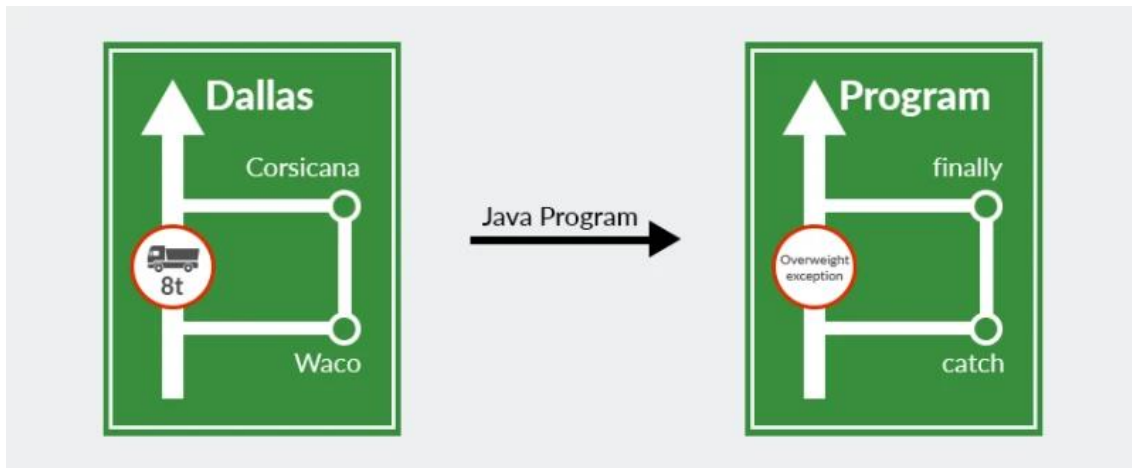
Aquí, la barra vertical equivale al or lógico. Se pueden añadir tantos tipos de excepciones como se quiera, separados por barras verticales. El significado es: “si se arroja una excepción del tipo tipoExcepcion1 o del tipo tipoExcepcion2 o ..., ejecutar este bloque catch, donde la excepción será referenciada con la variable e”.

Una estructura try-catch supone una bifurcación en el programa. Pero a menudo estamos interesados en que una serie de líneas de código se ejecuten, tanto si se produce una excepción como si no; por ejemplo, para cerrar un archivo en el que estábamos escribiendo.

Para esto se define un bloque opcional **finally** que, cuando está presente, se coloca al final una estructura try-catch (también es posible una estructura **try-finally**, sin bloque catch) y se ejecuta independientemente de si en el bloque try se ha lanzado una excepción o no, y de si la excepción ha sido capturada o no. Se ejecutará antes incluso que cualquier sentencia **return** que aparezca en los bloques **try** o **catch**. Esto nos asegura que, en circunstancias anómalas, se van a ejecutar determinadas tareas, como cerrar ficheros abiertos, antes de que termine la ejecución del programa. En el siguiente trozo de código:

```
try{  
    //blque que trabaja con archivos  
} catch (IOException e){  
    //bloque si salta excepción  
} finally {  
    //código para cerrar archivos  
}  
...  
return;
```

el bloque **finally** se ejecuta incluso antes de ejecutarse el **return** del bloque **try**, a pesar de que figura después de dicha sentencia, tanto si se produce una excepción como si no.



2.2. Propagación de excepciones

Todo el código en Java forma parte de algún método que, en última instancia, puede ser el método `main()`. Si desde un método `metodo2()` se invoca a un método `metodo1()` y salta una excepción durante la ejecución de este último, podemos capturarla y manipularla, como hemos hecho hasta ahora, con una estructura `try-catch` en el propio código de `metodo1()`. Pero hay un enfoque alternativo. Podemos declarar, en la definición de `metodo1()`, que en su ejecución puede producirse dicha excepción. Para ello usaremos la palabra clave **throws**.

```
tipo metodo1(tipo1 parametro, ...) throws tipoExcepcion{
    ...
}
```

En el cuerpo de `metodo1()` ya no tenemos que insertar los bloques `try-catch` para ese tipo de excepción. En cambio, `metodo2()` será el encargado de gestionarla cuando invoque a `metodo1()`.

```
tipo metodo2(tipo1 parametro, ...){
    ...
    try{
        metodo1();
    } catch (tipoExcepcion e) {
        //tratamiento de la excepción
    }
}
```

Veámoslo con un ejemplo. Supongamos que en método `metodo1()` se puede dar una división por cero. Por otra parte, un segundo método `metodo2()` llama a `metodo1()`. Lo que hemos hecho hasta ahora es:

```
void metodo1(int a, int b){
    int c;
    try {
        c = a/b;
    } catch(ArithmeticException e) {
        System.out.println("División por 0");
    }
    System.out.println(a + "/" + b + " = " + c);
}

void metodo2(){
    int x, y;
    metodo1(x,y);
}
```

Si la variable `y` es cero, la excepción producida es capturada y mantenida en el lugar donde se ha intentado la división (en `metodo1()`) antes de que la ejecución vuelva al método que lo llama (`metodo2()`).

Pero podríamos hacerlo de otra forma. Primero eliminamos el bloque `try-catch` de `metodo1()` y declaramos a este último como susceptible de producir una `ArithmeticException`. Esto se implementa por medio de la palabra clave `throws` en su encabezamiento.

```
void metodo1(int a, int b) throws ArithmeticException{
    int c;
    c = a/b;
    System.out.println(a + "/" + b + " = " + c);
}
```

Con esto estamos declarando que, dentro del método, puede producirse una excepción aritmética y que deberá ser manipulada por código externo, en el método que llame a `metodo1()`, en nuestro caso `metodo2()`. Además, esta particularidad, que forma parte de la definición de `metodo1()`, deberá constar en la documentación que la acompaña para que los usuarios del método sepan a

qué atenderse. La implementación de metodo2(), por tanto, deberá hacerse cargo de la excepción.

```
void metodo2(){
    int x, y;
    try{
        metodo1(x,y);
    } catch(ArithmeticException e) {
        System.out.println("División por cero");
    }
}
```

Por supuesto, metodo2() podría propagar la excepción a un tercer método que lo invoque, y así sucesivamente.

2.3. Lanzar excepciones

En Java, puedes lanzar una excepción usando la palabra clave "throw" seguida de una instancia de la excepción que deseamos lanzar. Hay varias formas de crear una instancia de una excepción en Java, pero la forma más común es crear una instancia de la excepción con el constructor de la excepción y pasar un mensaje de error como parámetro.

Además, debemos incluir en la declaración del método la palabra clave "throws" seguida del tipo de excepción que vamos a lanzar.

```
public class Operaciones {

    public static double dividir throws ArithmeticException(double
numerado, double denominador) {
        if (denominador == 0) {
            throw new ArithmeticException("El denominador no puede ser
cero.");
        }
        double resultado = numerador / denominador;
        return resultado;
    }
}
```

En este ejemplo, si el denominador es cero, se lanza una excepción `ArithmeticException` con el mensaje "El denominador no puede ser cero.". El programa se detendrá y no se imprimirá el resultado.

3. CREAR EXCEPCIONES

Hasta ahora, todas las excepciones que hemos visto vienen predefinidas en Java. Pero podemos implementar las nuestras propias con tan solo heredar de alguna que ya exista. Además, es posible lanzarlas cuando nos interese por medio de la palabra reservada **throw**.

Por ejemplo, si estamos introduciendo por teclado el valor entero que representa una edad, no tiene sentido un número negativo. Normalmente, en estos casos nos conformamos con un condicional y un simple mensaje de error. Pero también podemos crear con **new** y lanzar un **throw** una excepción definida por nosotros.

En la definición de una excepción de usuario no necesitamos implementar ningún método; basta con heredar de `Exception`. En todo caso, podemos sustituir el método `toString()` por uno que represente mejor nuestro caso.

```
public class ExcepcionEdadNegativa extends Exception{
    public String toString(){
        return "Edad negativa";
    }
}
```

Veamos un ejemplo donde se usa esta excepción:

```
try{
    System.out.print("Introducir edad: ");
    int edad = new Scanner (System.in).nextInt();
    if (edad < 0){
        throw new ExcepcionEdadNegativa();
    }
    else {
        System.out.println("edad correcta: " + edad)
    }
} catch (ExcepcionEdadNegativa ex){
    e.printStackTrace();
}
```

A la hora de hacer nuestras excepciones, hay que conocer algunos métodos de la clase `Exception`, por si queremos sobrescribirlos:

- **`Exception()`**: es el constructor por defecto.
- **`Exception(String message)`**: constructor al que se le pasa un mensaje de error.
- **`toString()`**: método de la clase **`Object`** redefinido por **`Throwable`** para que muestre el nombre de la clase y si tiene mensaje, lo mostrará también.
- **`getMessage()`**: método heredado de la clase **`Throwable`** que nos devuelve una cadena que contiene el mensaje con el que fue creada la excepción.
- **`printStackTrace()`**: método heredado de la clase **`Throwable`** que directamente imprime a la salida estándar de errores el objeto de la clase desde el que se invoca junto con la traza de llamadas a métodos desde que se ha producido la excepción. Resulta bastante útil para depurar programas y así encontrar el lugar exacto del fallo.

4. ASERCIONES

Una aserción es una condición lógica insertada en el código Java, de ideas o condiciones que se asumen que son ciertas. El sistema se encarga de comprobarlas y avisar mediante una excepción en caso de que no se cumplan.

Generalmente se suele utilizar para verificar valores de las variables en cierto punto del programa.

Se aconseja su uso durante el desarrollo y las pruebas, y se pueden eliminar de la implantación del sistema. Concretamente en estas situaciones:

- Usarse en la entrada de métodos privados.
- Usarse en la salida de métodos públicos o privados.
- Usarse para verificar cómo se supone que están las variables y estructuras de datos internas.
- Usarse en la sentencia **`default`** de la estructura **`switch`** cuando todos los casos correctos están explícitos (es decir, cuando no debería ocurrir).

- Usarse en el último **else** de construcciones **if ... else if ... switch** cuando todos los casos correctos están explícitos (es decir, cuando el último **else** no debería ejecutarse nunca).
- Usarse en bucles largos.

No se recomienda su uso en estas situaciones:

- Usarse para detectar errores en los datos de entrada de los programas.
- Usarse en la entrada de métodos públicos.

Para incluir aserciones a nuestros códigos podemos utilizar dos tipos de sintaxis.

La primera sintaxis sería la siguiente:

```
assert expresion;
```

Por ejemplo:

```
double m = 100.0;  
double n = m/2;  
assert m == n*2; //condición que asumimos como true
```

La segunda opción de sintaxis es la siguiente:

```
assert expresion1:expresion2;
```

Por ejemplo:

```
double m = 100.0;  
double n = m/2;  
assert m == n*2 : "Java no sabe dividir " + m + " entre 2: " + n;
```

Si te das cuenta, la sintaxis es similar a la del operador ternario, el cuál ya vimos en su momento. La expresión entre “assert” y “:” se evalúa y en caso de que no se cumpla se ejecutará la expresión que va detrás de “:”.

Para entenderlo mejor, veamos una comparación de dos códigos que harían lo mismo:

```
if(!estado_ideal){  
    throw new Error("Fallo en esta zona del programa");  
}
```

Es equivalente a:

```
assert estado_ideal : "Fallo en esta zona del programa";
```

Si queremos que nos salte una excepción en el caso de que el siguiente for se ejecute completamente podríamos hacer lo siguiente:

```
for(int i = 0; i < 10; i++){  
    if(array[i] > 1000){  
        return i;  
    }  
}  
assert false;
```

En el caso de que el bucle termine sin haber ejecutado el return, se evaluará la expresión de assert (en este caso false) por lo que lanzará una excepción (AssertionError).

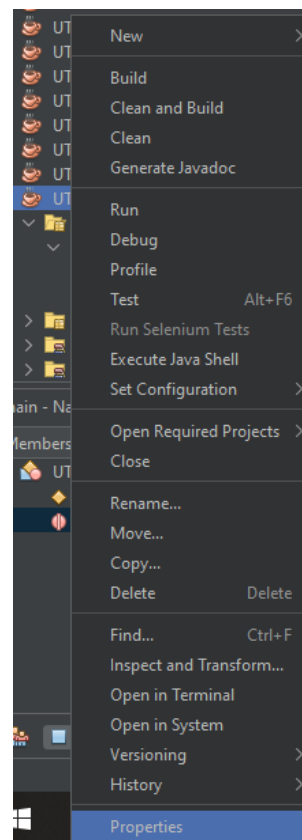
Otro de los casos que nos recomiendan es controlar cuando el default de un switch nunca debería ocurrir, por ejemplo:

```
switch(op){  
    case 1: sumar();  
            break;  
    case 2: restar();  
            break;  
    case 3: multiplicar();  
            break;  
    case 4: dividir();  
            break;  
    default: assert false;  
}
```

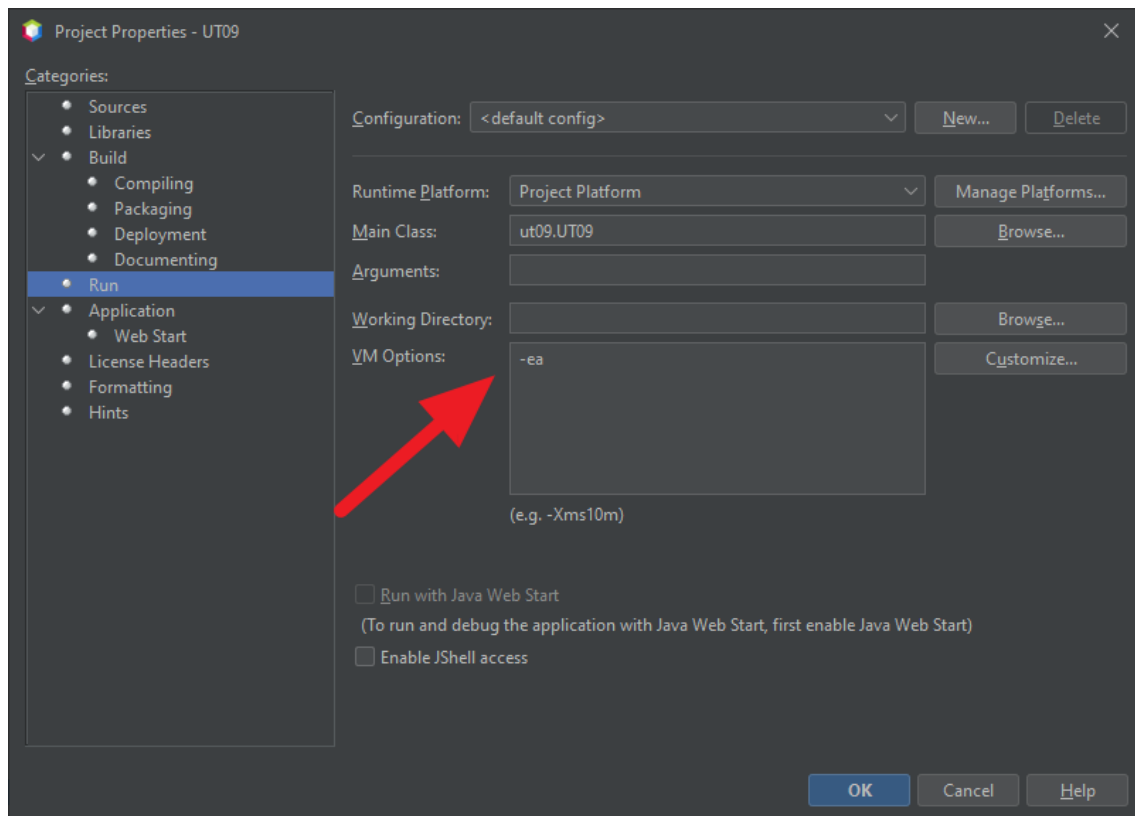
En el caso de no recibir un 1, 2, 3 o 4 nos devolverá una excepción.

4.1. Activar y desactivar las aserciones

Las aserciones se encuentran desactivadas por defecto, por tanto, para poder utilizarlas vamos a necesitar activarlas. Solo tendremos que pulsar botón derecho del ratón sobre el proyecto y seleccionar “properties”...



Después iremos a la sección Run y en el cuadro de texto VM Options escribiremos “-ea”.



Cuando queramos desactivar las aserciones solo tendremos que volver a quitar el “-ea”.